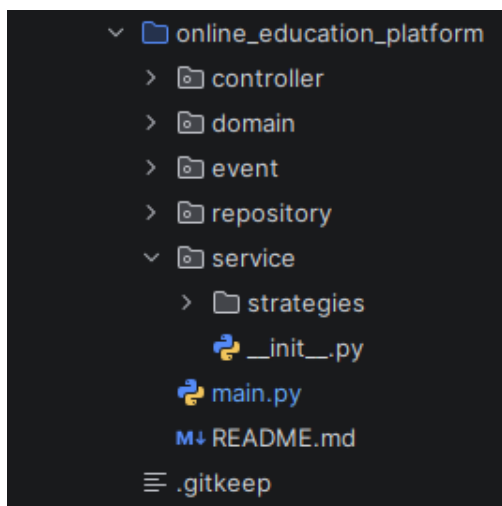


مستندات ساخت و استقرار

در این فاز معماری طراحی شده در مرحله قبل به یه نمونه اولیه قابل اجرا تبدیل شد. هدف ما این بود که پیاده سازی ساختارمون تا حد امکان با نمودار DCD و معماری انتخاب شده در فاز طراحی منطبق باشه تا علاوه بر اینکه بتونیم سناریوهای اصلی سیستم رو اجرای کنیم، توسعه و نگهداری پروژه هم حفظ بشه. پس قبل از شروع کدنویسی، ساختار پوشه ها، مسئولیت هر لایه و نحوه ارتباط بین اجزای سیستم رو مشخص کردیم و بعد پیاده سازی رو بر اساس همون ساختار انجام دادیم.

ساختار پروژه

با توجه به اینکه در فاز طراحی از معماری لایه ای در کنار اصول Domain Driven Design استفاده کرده بودیم پروژه هم بر همین اساس سازمان دهی شد. هر بخش فقط مسئول انجام وظایف مربوط به خودش هست و وابستگی بین لایه ها به حداقل رسیده.



داخل این ساختار کلاس های دامنه تنها شامل موجودیت ها و قوانین اصلی کسب و کار هستن و هیچ وابستگی مستقیمی به پایگاه داده یا رابط کاربری ندارن. سرویس ها منطق اجرایی سیستم رو پیاده سازی می کنن، Repository هم مسئول مدیریت داده ها هستش، Controller درخواست های ورودی رو مدیریت می کنه و بخش Event هم وظیفه مدیریت رویدادها و ارسال اعلان ها رو به عهده داره.

محیط توسعه

برای پیاده سازی نمونه اولیه زبان Python رو انتخاب کردیم چون توسعه ساده میشد، خوانایی بالا می رفت، اعضای تیم به اون تسلط کافی رو داشتن و امکان پیاده سازی سریع الگوهای طراحی شی گرا رو ممکن می کرد.

محیط توسعه پروژه به صورت زیره:

- زبان برنامه نویسی Python 3
- معماری شی گرا
- اجرای پروژه از طریق Python Interpreter
- عدم نیاز به نصب پایگاه داده
- استفاده از Repository های In-Memory برای نگهداری داده ها در حافظه

داخل این نسخه از پروژه تمام اطلاعات در حافظه اصلی نگهداری میشن تا تمرکز پروژه روی معماری نرم افزار و نحوه تعامل اجزای سیستم باشه. برای نسخه نهایی این لایه می تونه بدون تغییر سایر قسمت ها با یه پایگاه داده واقعی جایگزین بشه

فرآیند Build

از اونجایی که پروژه با زبان Python توسعه داده شد فرآیند Build پیچیدگی خاصی نداره و طبیعتاً نیازی به مرحله کامپایل وجود نداره. بعد از دریافت سورس پروژه فقط کافیه ساختار پوشه ها حفظ و فایل اصلی پروژه اجرا بشه.

فرآیند ساخت شامل مراحل زیر میشه:

- دریافت سورس پروژه
- قرار گرفتن فایل ها در ساختار تعریف شده
- ایجاد نمونه Repository ها
- ایجاد سرویس های دامنه
- تزریق وابستگی ها
- مقداردهی اولیه سیستم
- اجرای سناریوی آزمایشی

در فایل main.py تمامی این مراحل به ترتیب انجام شده.

مدیریت وابستگی ها

یکی از مهم ترین نکاتی که هنگام پیاده سازی رعایت کردیم جلوگیری از وابستگی مستقیم کلاس ها به هم بود. برای این جلوگیری از اصل Dependency Injection استفاده کردیم.

به عنوان مثال کلاس ExamService مستقیماً Repository ها رو ایجاد نمی کنه بلکه Repository های مورد نیاز هنگام ایجاد شی به اون تزریق می شن

این روش باعث شد که وابستگی بین کلاس ها کاهش پیدا کنه، تست کردن سیستم ساده تر بشه، بعداً بتونیم نوع Repository یا پایگاه داده رو بدون تغییر منطق سیستم جایگزین کنیم. همین رویکرد در سرویس های CourseService و NotificationService نیز رعایت شده است.

پیاده‌سازی لایه دامنه

در لایه Domain تمامی موجودیت‌های اصلی سامانه آموزشی پیاده‌سازی شدن که شامل Teacher ، Student ، User ، Grade ، Submission ، Question ، Exam ، ClassSession ، Enrollment، Content، Course، Admin ، Notification هستند و مطابق نمودار طراحی تمامی موجودیت‌ها از کلاس BaseEntity ارث‌بری می‌کنن. شناسه یکتای هر شی با استفاده از UUID تولید میشه. این موضوع باعث می‌شه که وابستگی به نوع خاصی از پایگاه داده وجود نداشته باشه و هر موجودیت دارای شناسه یکتا و مستقل باشه.

پیاده‌سازی Repository

برای نگهداری اطلاعات الگوی Repository مطابق طراحی پیاده‌سازی شد. برای هر موجودیت اصلی یه Interface مستقل تعریف شد و سپس نسخه In-Memory آن پیاده‌سازی شد.

نمونه ای از Repository هایی که ایجاد کردیم:

UserRepository -

CourseRepository -

EnrollmentRepository -

ExamRepository-

SubmissionRepository -

NotificationRepository -

ClassSessionRepository -

داخل این نسخه داده‌ها داخل Dictionary نگهداری میشن اما طراحی Interface ها جوری انجام شده که در آینده بتونیم بدون تغییر سایر قسمت‌های سیستم Repository ها رو به پایگاه داده‌هایی مثل PostgreSQL یا MySQL متصل کنیم.

پیاده‌سازی منطق کسب‌وکار

منطق اصلی سیستم در لایه Service قرار گرفته. در این بخش سرویس‌های زیر رو توسعه دادیم:

- CourseService

- ExamService

- GradeService

- NotificationService

این سرویس‌ها مسئول اجرای فرآیندهایی مثل ثبت نام دانشجو، ایجاد آزمون، ارسال پاسخ، محاسبه نمره، مدیریت اعلان‌ها و ارائه محتوای آموزشی هستند. قرار گرفتن این منطق در Service باعث شد موجودیت‌های دامنه تنها مسئول نگهداری داده و قوانین اصلی باشن و پیچیدگی سیستم کاهش پیدا کنه.

استفاده از Design Pattern ها

در فرآیند پیاده‌سازی الگوهای طراحی انتخاب شده در فاز معماری نیز به‌صورت کامل پیاده‌سازی شدن.

الگوی Repository برای جداسازی منطق ذخیره‌سازی داده‌ها استفاده شد.

الگوی Strategy برای پیاده‌سازی روش‌های مختلف تصحیح آزمون به کار رفت و امکان انتخاب روش تصحیح خودکار یا دستی را فراهم کرد.

برای اعمال جریمه تاخیر از Decorator استفاده کردیم تا بدون تغییر منطق اصلی تصحیح درصد جریمه به نمره اضافه بشه.

برای مدیریت اعلان‌ها و رویدادهای سیستم از الگوی Observer استفاده کردیم تا انتشار رویدادها و واکنش سرویس‌های مختلف به صورت مستقل انجام بشه.

استقرار (Deployment)

از اونجا که هدف این فاز ارائه نمونه اولیه بود سیستم به صورت Local اجرا شد و نیازی به استقرار روی سرور وجود نداشت. فرآیند استقرار شامل مراحل زیر شد:

- دریافت سورس پروژه

- اجرای فایل main.py

- ایجاد تمامی Repository ها

- ایجاد سرویس ها

- ایجاد داده های اولیه

- اجرای سناریوی تست

در سناریوی آزمایشی یه استاد و یه دانشجو ایجاد میشن و یه درس ثبت می شه. دانشجو در درس ثبت نام می کنه، آزمون ایجاد و منتشر می شه، دانشجو پاسخ های خودش رو ارسال می کنه و در نهایت سیستم نمره، وضعیت قبولی و اطلاعات ورود کاربر را نمایش می دهد. این سناریو نشان می دهد که اجزای مختلف سامانه مطابق طراحی اولیه با یکدیگر تعامل دارند.

تطابق پیاده سازی با نمودار طراحی

در طول پیاده سازی تلاش کردیم ساختار پروژه با نمودار کلاس طراحی (DCD) کاملاً منطبق باشد. موجودیت ها، سرویس ها، Repository ها، کنترلرها و Event ها مطابق کلاس های تعریف شده در نمودار ایجاد شدن و ارتباط میان آنها نیز بر اساس وابستگی های مشخص شده در طراحی پیاده سازی شد. همچنین استفاده از UUID ، تزریق وابستگی، الگوهای طراحی Repository ، Strategy ، Decorator و Observer و جداسازی مسئولیت لایه ها، همگی مطابق معماری انتخاب شده در فازهای قبلی انجام شدن. به همین دلیل نمونه اولیه تولید شده نه تنها سناریوهای اصلی سامانه آموزشی را اجرا می کنه بلکه از نظر ساختاری نیز قابلیت توسعه، نگهداری و اتصال به زیرساخت های واقعی در مراحل بعدی پروژه رو داره .

ساختار نهایی پروژه در یک نگاه:

```

└─ online_education_platform
  └─ controller
    ├── __init__.py
    ├── app_controller.py
    └── exam_controller.py
  └─ domain
    ├── __init__.py
    ├── base_entity.py
    ├── class_session.py
    ├── course.py
    ├── enrollment.py
    ├── exam.py
    ├── notification.py
    └── user.py
  └─ event
    ├── __init__.py
    ├── event_listeners.py
    └── event_publisher.py
  └─ repository
    ├── __init__.py
    ├── in_memory_repo.py
    └── interfaces.py

└─ service
  └─ strategies
    ├── auto.py
    ├── base.py
    ├── course_service.py
    ├── exam_service.py
    ├── grade_service.py
    ├── late_penalty.py
    ├── manual.py
    └── notification_service.py
  ├── __init__.py
  ├── main.py
  └─ README.md
```